National Taiwan Normal University
CSIE Computer Programming I

Instructor: Po-Wen Chi
Due Date: 2024.11.19 PM 11:59

Assignment 3

Policies:

- Zero tolerance for late submission.
- Please pack all your submissions in one zip file. **RAR is not allowed!!**
- For convenience, your executable programs must be named following the rule hwXXYY, where the red part is the homework number and the blue part is the problem number. For example, hw0102 is the executable program for homework #1 problem 2.
- I only accept **PDF** or **TEXT**. MS Word is not allowed.
- Do not forget your Makefile. For convenience, each assignment needs only one Makefile.
- Please provide a README file. The README file should have at least the following information:
  - Your student ID and your name.
  - How to build your code.
  - How to execute your built programs.
  - Anything that you want to notify our TAs.
- **DO NOT BE A COPYCAT!!** You will get ZERO if you are caught.

3.1 Riemann (20 pts)

In mathematics, an integral assigns numbers to functions in a way that can describe displacement, area, volume, and other concepts that arise by combining infinitesimal data. I believe that you know how to calculate the definite integral of a polynomial, right? If not, you can ask TA for help.

When studying integral, I believe that your teacher has shown you **Riemann integral**, an integral definition. Given a polynomial $f(x)$, you can calculate the Riemann sum of $f(x)$ in the interval $[a, b]$ as follows.

$$\sum_{i=0}^{n-1} f(a + \frac{b-a}{n} \times i) \times \frac{b-a}{n}.$$

This time, I want you to develop a Riemann Sum library. For your simplicity, you only need to consider the case of polynomial functions with order 2. The function declarations are as follows.

```
// Setup the function as f(x) = ax^2 + bx + c
void set_polynomial( int32_t a, int32_t b, int32_t c );

// Calculate Riemann Integral.
// The interval is between begin and end.
// For your simplicity, I promise that begin <= end.
double calculate_riemann_integral( int32_t begin, int32_t end );

// Calculate Riemann Sum.
// The interval is between begin and end.
// n is the partition number
// For your simplicity, I promise that begin <= end.
double calculate_riemann_sum( int32_t begin, int32_t end,
    int32_t n );

// Calculate Area covered between f(x) and x-axis.
// The interval is between begin and end.
// For your simplicity, I promise that begin <= end.
double calculate_area( int32_t begin, int32_t end );

// Calculate Area covered between f(x) and x-axis with Riemann
    sum.
// The interval is between begin and end.
// n is the partition number
// For your simplicity, I promise that begin <= end.
double calculate_area_riemann_sum( int32_t begin, int32_t end,
    int32_t n );
```

You should also prepare a header file called riemann.h. Our TAs will prepare hw0301.c which includes riemann.h and uses these functions. **Do not forget to make hw0301.c to hw0301 in your Makefile.** For your simplicity, I promise that TA will call **set_polynomial** before using all other functions.

3.2 Gacha (20 pts)

Have you ever played mobile games? Nowadays, Gacha is the most interesting (evil) part of mobile games. This time, I want you to develop a Gacha related library. Suppose there are six different groups of cards that

a player can get. P_i denotes the probability that a user gets i-group card. Note that

$$P_1 < P_2 < P_3 < P_4 < P_5 < P_6.$$

The functions are defined as follows.

```
// Initialize Gacha environment
// n is the number of cards
// The card ID will be 1 to n
void initialize( int32_t n );

// Set group size
// Group 1: 1 -- g1
// Group 2: g1+1 -- g2
// Group 3: g2+1 -- g3
// Group 4: g3+1 -- g4
// Group 5: g4+1 -- g5
// Group 6: g5+1 -- n
// Undoubtedly, g1 < g2 < g3 < g4 < g5 < n
// For any invalid inputs, return -1; otherwise, return 0.
int32_t set_groups( int32_t g1, int32_t g2, int32_t g3, int32_t
    g4, int32_t g5 );

// Set group probability
// Group 1: p1
// Group 2: p2
// Group 3: p3
// Group 4: p4
// Group 5: p5
// Group 6: 1 - p1 - p2 - p3 - p4 - p5
// Undoubtedly, p1 < p2 < p3 < p4 < p5 < 1 - p1 - p2 - p3 - p4
   - p5
// For any invalid inputs, return -1; otherwise, return 0.
int32_t set_props( double p1, double p2, double p3, double p4,
   double p5 );

// Set guaranteed counter.
// Every count pull times, you will get at least one card from
   group 1.
// Once a user get a card from group 1, the counter will be
   reset.
// For any invalid inputs, return -1; otherwise, return 0.
int32_t set_guarantee( int32_t count );

// Add/Get the left pull number.
// Initially the pull number is 0.
// Return:
// add --> For the invalid inputs, return -1; otherwise, return
    0.
// get --> Return the left pull number
int32_t add_pull_number( int32_t new );
```

```
int32_t get_pull_number( void );

// Randomly print 1/10 card IDs from the given pool.
// Note that pull10 must guarantee at least one group 3 or better card.
// Each pull will cost one pull number.
// If the initial position is not valid or the left pull number is not enough, return -1.
// Otherwise, return 0.
int32_t pull();
int32_t pull10();
```

You should also prepare a header file called gacha.h. Our TAs will prepare hw0302.c which includes gacha.h and uses these functions. **Do not forget to make hw0302.c to hw0302 in your Makefile.**

3.3 Euclidean Algorithm (20 pts)

In mathematics, the Euclidean algorithm, or Euclid's algorithm, is an efficient method for computing the greatest common divisor (GCD) of two integers (numbers), the largest number that divides them both without a remainder. This time, I want you to develop a function to not only get GCD of two integers, but also **print** the process.

```
// Get the GCD of a and b. a should be greater than b
// print_format:
//     0: No print
//     1: mathematical expression
//     2: vertical expression
// If a < b or invalid print_format, return 0; otherwise, return gcd
uint32_t gcd( int32_t print_format, uint32_t a, uint32_t b );

/*
Here are the output example. I use 159 and 36 for example.

Mathematical expression:
159 / 36 = 4 ... 15
36 / 15 = 2 ... 6
15 / 6 = 2 ... 3
6 / 3 = 2 ... 0

Vertical expression (alignment is important):
4 |159| 36| 2
  |144| 30|
  ---------
2 | 15|  6| 2
  | 12|  6|
  ---------
  |  3|  0|
```

```
26 */
```

You should also prepare a header file called mygcd.h. Our TAs will prepare hw0303.c which includes mygcd.h and uses these functions. **Do not forget to make hw0303.c to hw0303 in your Makefile.**

3.4 Tower of Hanoi (20 pts)

The Tower of Hanoi is a mathematical game or puzzle. It consists of three rods and a number of disks of different sizes, which can slide onto any rod. The puzzle starts with the disks in a neat stack in ascending order of size on one rod, the smallest at the top, thus making a conical shape. Figure 3.1 is an example with 8 disks.

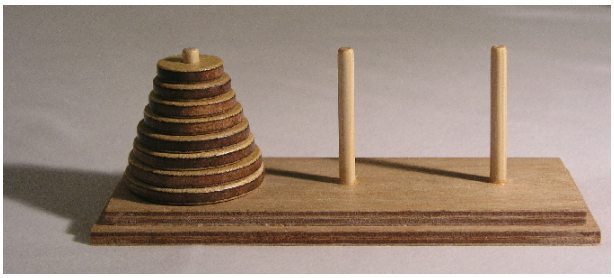

FIGURE 3.1: Tower of Hanoi

The objective of the puzzle is to move the entire stack to another rod, obeying the following simple rules:

- Only one disk can be moved at a time.
- Each move consists of taking the upper disk from one of the stacks and placing it on top of another stack or on an empty rod.
- No larger disk may be placed on top of a smaller disk.

Please write a program to list the procedure of moving n disks from one rod to another. We assume there are only 3 rods, and all disks are placed on the first rod with the ascending order of size. Our target is to move these n disks from the first rod to the second rod. The disks are labeled $1, 2, \ldots, n$ from top to bottom.

Note that the Hanoi problem is a very famous recursive problem. For your convenience, I give you a hint as follows:

- Move $m-1$ disks from the source to the spare rod, by the same general solving procedure. Rules are not violated, by assumption. This leaves the disk m as a top disk on the source rod.

- Move the disk m from the source to the target rod, which is guaranteed to be a valid move, by the assumptions.
- Move the $m-1$ disks that we have just placed on the spare, from the spare to the target rod by the same general solving procedure, so they are placed on top of the disk m without violating the rules.

Wait! I have told you that every recursive program can be converted to an iterative program (loop). So this time, I ask you to write **TWO** programs. hw0304-1 is the **recursive** version and hw0304-2 is the **iterative** function. Good Luck.

```
$ ./hw0304-1
Please enter the disk number (2-20): 2
move disk 1 to rod 3
move disk 2 to rod 2
move disk 1 to rod 2
```

You need to implement two different functions in another C code and prepare a header file.

3.5 Shapez.io (20 pts)

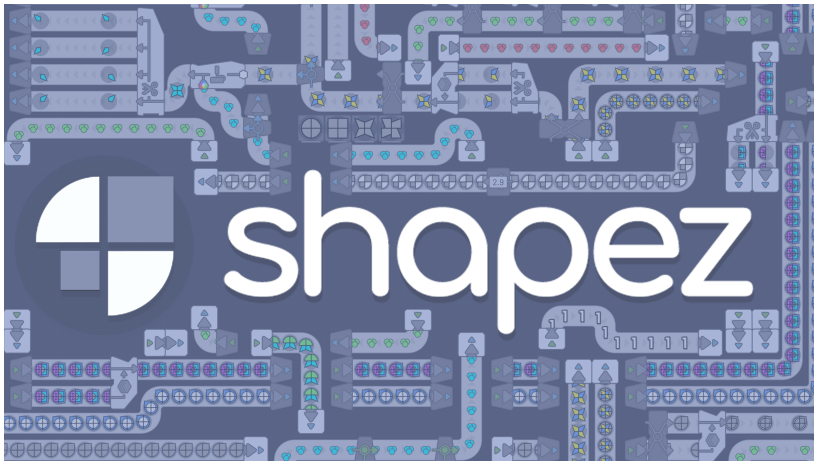

FIGURE 3.2: shapez.io

Do you enjoy games focused on automation and production?

Shapez.io is a casual game about building factories, automating production, and combining shapes.

You can purchase the single-player version of the game on Steam, or try the free demo in web browsers : shapez.io !

Shapez - Launch Trailer

Game Overview

Shapes or products in this game are extracted using mining machines. Each shape is a geometric figure that can be divided into four equal parts.

In the game, you can use conveyor belts to transport items from one machine to another.

By rotating, cutting, combining, coloring, and mixing, you can transform basic resources into the target shapes to complete tasks and send shapes to the core.

Shapes

There are four types of shapes in this game:

- 0 - - - No shape
- 1 - C - Circle
- 2 - R - Rectangle
- 3 - W - Windmill / Fan
- 4 - S - Star

The game includes eight colors:

- 0 - - - No shape
- 1 - r - Red
- 2 - g - Green
- 3 - b - Blue
- 4 - y - Yellow
- 5 - p - Purple
- 6 - c - Cyan
- 7 - u - Uncolored
- 8 - w - White

Each segment can have an independent shape and color, represented by a string of codes.

The shapez is a special INT , can translated mapping into only one of the shapes code .

For example the number 18181818 can represent the shapes codes CwCwCwCw . (and this is for simplified , the actual translate function is be encrypted in TA's Function below)

You can convert codes to shapes on this site: https://viewer.shapez.io/. Only single-layer shapes will appear in this assignment.

Machines

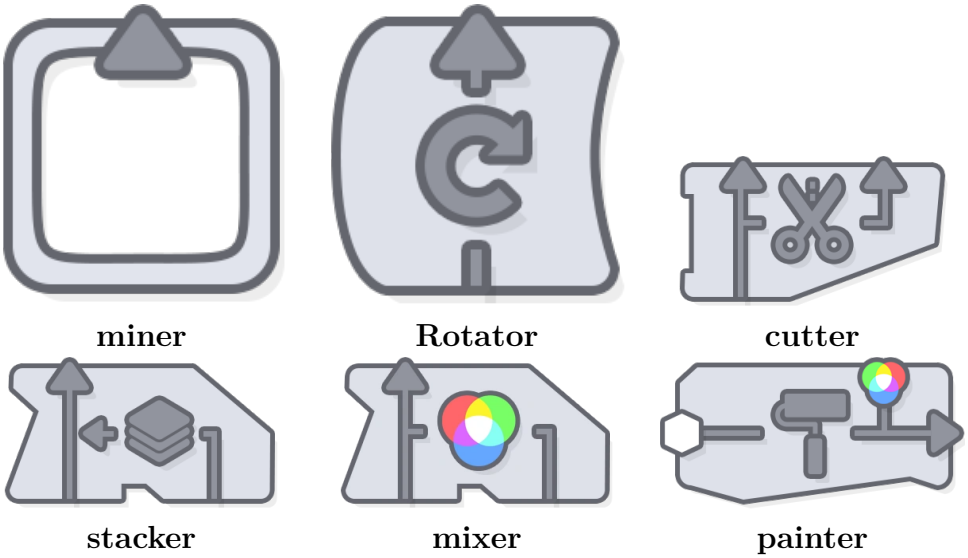

TABLE 3.1: machines

Miner

Extracts a complete Uncolored shapez and returns one shapez(INT). Only allows input of 'C', 'R', 'W', 'S'(1,2,3,4).

```
int miner(int shape);
```

Rotator

Rotates shapes clockwise 90 degree and returns one shapez(INT).

```
int rotator(int shapez);
```

Cutter

Cuts shapes and returns one shapez(INT). in this assignment, it retains only the left half of the shape.

```
int cutter(int shapez);
```

Stacker

Combines two shapez(INT) into and returns one shapez(INT).. Shapes cannot overlap.

```
int stacker(int shapezA, int shapezB);
```

Mixer

Mixes two colors based on the chart below. Only allows input of ‘r‘, ‘g‘, ‘b‘ (1,2,3) or return values.

```
int mixer(int colorA, int colorB);
```

	Red	Green	Blue	Cyan	Yellow	Magenta
Red	Red	Yellow	Magenta	White	Yellow	Magenta
Green	Yellow	Green	Cyan	Cyan	Yellow	White
Blue	Magenta	Cyan	Blue	Cyan	White	Magenta
Cyan	White	Cyan	Cyan	Cyan	White	White
Yellow	Yellow	Yellow	White	White	Yellow	White
Magenta	Magenta	White	Magenta	White	White	Magenta

FIGURE 3.3: Color Mixer Chart

Painter

Overrides the color on every part of shapez(INT). Only allows input of ‘r‘, ‘g‘, ‘b‘(1,2,3) or return values.

```
int painter(int shapez, int color);
```

Support Functions by TA

You can use those useful functions by TA in shapez.h

You can load_shapez at first and return store_shapez at end, this can make registers only mutable in functions.

Registers

Registers are a set of global variables you can use those data in functions.

```
int s1, s2, s3, s4; // shapes
int c1, c2, c3, c4; // colors

int tmpS, tmpC; // temp var
int tmpZ; // temp shapez
```

load_shapez

Loads the shapez(INT) into registers.

```
void load_shapez(int shapez);
```

store_shapez

Stores registers into a shapez(INT).

```
int store_shapez();
```

print_shapez

Prints the shape code on the terminal.
You can convert codes to shapes on this site: https://viewer.shapez.io/.
It's for you to debug, don't use this if you finished test.

```
void print_shapez(int shape);
```

Rules

- Direct doing math operation on shapez and declaring any variables when implementing machines is forbidden; only the variables (registers) in `shapez.c` can be used.
- When implementing the Target function, you can only use machines or other functions, without using variables.
- You can implement other functions yourself (for example : packageing 3 Rotator to make counterclockwise rotation machine), but only machines or other functions may be used, without any variable declarations.

Target

Finish those 10 task (from 1 to 10) using machines or other functions in this function, return the shapez(INT).

```
int Target(int task);
```

Tasks:

```
CwCwCwCw
--RbRb--
CwCwCwRb
RuCwRuCw
CcSpSpCc
Sr--Sb--
CwRp--Sc
SySrWrSc
RpRuWrSu
SgWy--Cr
```

There are 10 more tasks which the TA will implement by combining your functions, so trying to make your function Pure (no side effect) and work functional. :)

Note

You should also prepare a implantation file called shapez.c. Our TAs will prepare hw0305.c encrypt.h encrypt.c shapez.h hw0305 will include shapez.h and encrypt.h and uses these functions.Do not forget to make hw0305.c to hw0305 in your Makefile.

3.6 Bonus (5 pts)

In this class, I have warned you that you need to add **-lm** when building your code if you use standard math library. However, I also show you the case that the following code can be built without **-lm** parameter. Please give the reason and design an experiment to show it.

```
#include <stdio.h>
#include <math.h>

int main()
{
    printf( "sqrt(%f) = %f\n", 900.0, sqrt( 900.0 ) );
    printf( "cbrt(%f) = %f\n", -27.0, cbrt( -27.0 ) );
    printf( "pow(%f, %f) = %f\n", 2.0, 7.0, pow( 2.0, 7.0 ) );
    printf( "sin(%f) = %f\n", 0.0, sin( 0.0 ) );
    printf( "cos(%f) = %f\n", 0.0, cos( 0.0 ) );
    printf( "tan(%f) = %f\n", 0.0, tan( 0.0 ) );

```

```
13      return 0;
14  }
```